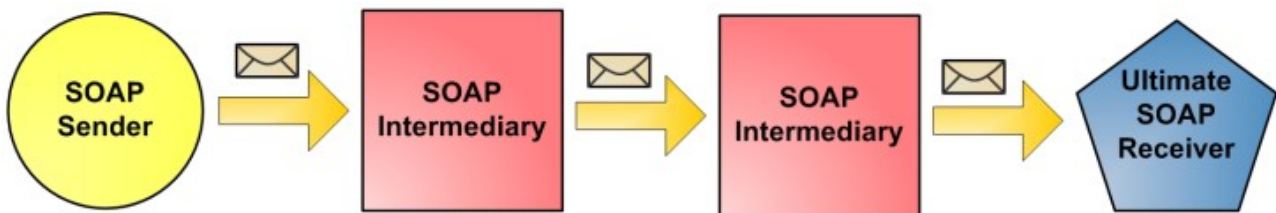


3 Developing Web Services Using SOAP

3.1.1 SOAP (Simple Object Access Protocol)

SOAP Message Transmission

- Message delivery path involves intermediaries
- The SOAP Message Transmission involves three main roles:
 - The SOAP Sender creates and sends a SOAP Message to an ultimate SOAP Receiver
 - One or more optional SOAP Intermediaries can be positioned to intercept messages between the the sender and the receiver. They can perform filtering, logging, caching etc.
 - The SOAP sender's intended destination is called the Ultimate SOAP Receiver



3.1.2 WSDL (Web Services Description Language)

What is WSDL?

- WSDL is an XML document containing information about web services such as method name, method parameter and how to access it
- It defines a binding mechanism used to attach a protocol, data format, an abstract message, or set of endpoints defining the location of services
- In the core of the Web services model
 - WSDL is used as the metadata language for defining Web services and describes how service providers and requesters communicate with one another
 - WSDL describes the Web services functionalities offered by the service provider, where the service is located, and how to access the service

3.1.3 Universal Description, Discovery, and Integration (UDDI)

- An XML-based framework for describing, discovering and integrating web services
- UDDI is a directory of web service interfaces described by WSDL, containing information about web services
- Web services brokers use UDDI as a standard for registering the Web service providers

- By communicating with the UDDI registries, the service requesters locate services and then invoke them

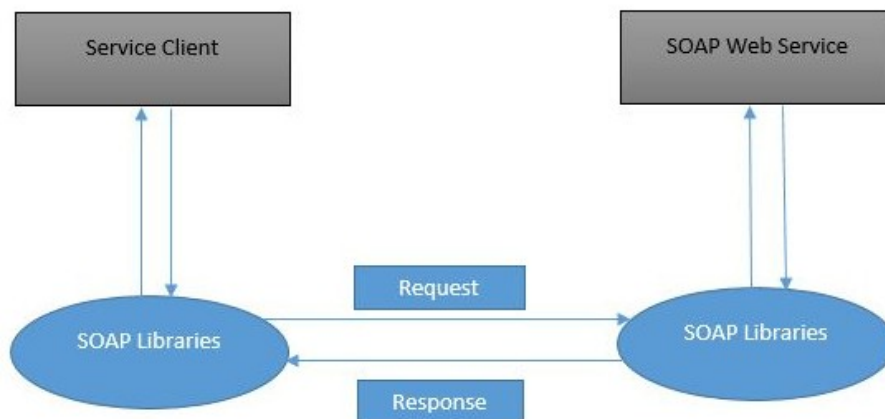
Readings

- Martin Kalin. Java Web Services: Up and Running

3.2 SOAP-based Web Services Implementation

3.2.1 Typical Scenario – Request/response message exchange pattern

- The client's underlying SOAP library sends a SOAP message as a service request
- The web service's underlying SOAP library sends another SOAP message as the corresponding service response



3.2.2 Implementation

A SOAP-based web service

- Can be implemented as a single Java class
- But for best practices, there should be
 - An interface
 - Also called the SEI: Service Endpoint Interface
 - The interface declares the methods, which are the web service operations
 - An implementation
 - Also called the SIB: Service Implementation Bean
 - Defines the methods declared in the interface

- Can be either a POJO (Plain Old Java Object) or a Stateless Session EJB (Enterprise Java Bean)
 - POJO: Instances of a regular Java class
 - EJB: A specification provided to develop secured, robust and scalable distributed applications; we'll use EJB to implement web service that runs in an Application Server

3.2.3 JAX-WS (Built-in Java SE)

- For simple SOAP-based web services, Java SE includes a built-in HTTP server that requires no external libraries
- We shall use this server in our earlier examples, then use the glassfish server in later examples
- Steps
 - Annotate
 - Add annotation `@WebService` or `@WebServiceProvider` to the implementation class
 - Add annotation `@WebMethod` to the web service's operations
 - Publish: Use the `Endpoint.publish(address, implementor)` method in the main method
 - Syntax for publishing
 - `Endpoint.publish("http://localhost:port#/service", new MyWebService());`

3.3 Some examples

3.3.1 Example 1: TimeServer, RPC Style

- We start with a simple, local TimeServer program that returns the current time as either a string or as the elapsed milliseconds from the Unix epoch, midnight January 1, 1970 GMT.
- Then, we create a corresponding web service.

3.3.1.1 Local Time Server

TimeServer1Interface.java – a simple interface

```
package NotWS;
public interface TimeServer1Interface {
    String getTimeAsString();
    long getTimeAsElapsed();
}
```

TimeServer1Implementation.java – implementation of the interface functions

```
package NotWS;
import java.util.Date;
public class TimeServer1Implementation implements TimeServer1Interface {
    @Override
    public String getTimeAsString() {
        return new Date().toString();
    }
    @Override
    public long getTimeAsElapsed() {
        return new Date().getTime();
    }
}
```

TimeServer1.java – Client for the time server

```
package NotWS;
public class TimeServer1 {
    public static void main(String[] args) {
        TimeServer1Implementation ts = new TimeServer1Implementation();
        System.out.println(ts.getTimeAsString());
        System.out.println(ts.getTimeAsElapsed());
    }
}
```

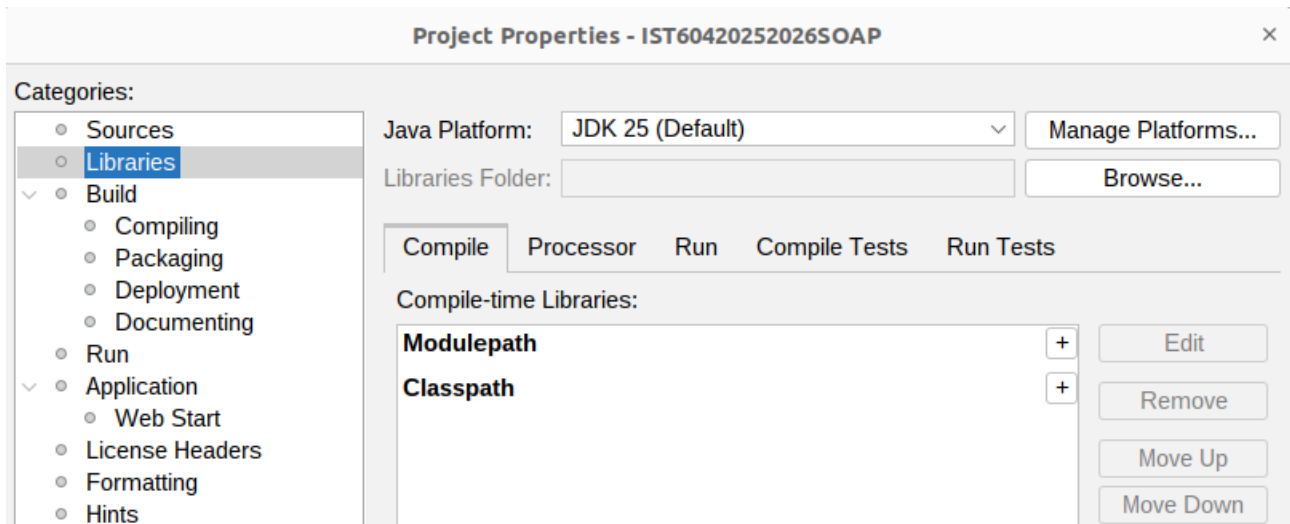
Output

```
Sat Apr 11 06:45:04 WAT 2026
1775886304269
```

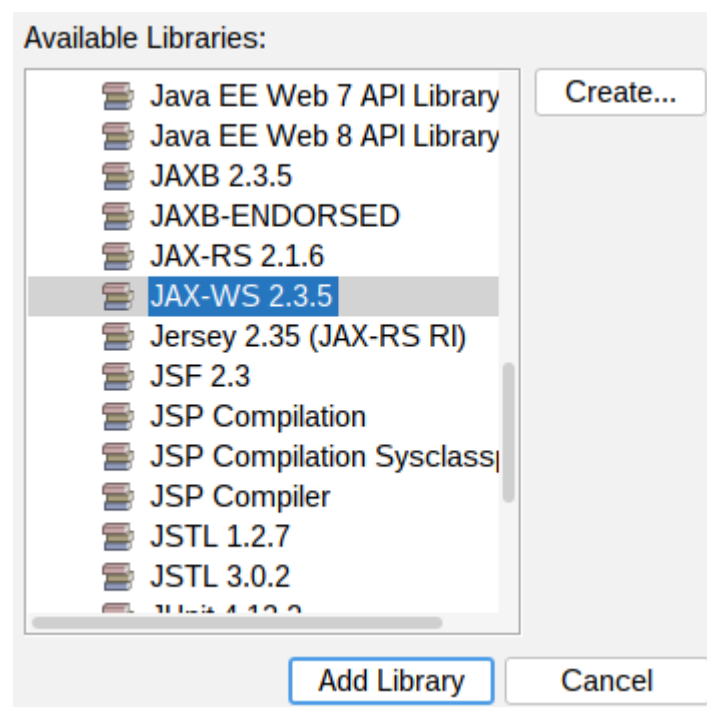
3.3.1.2 Web Service

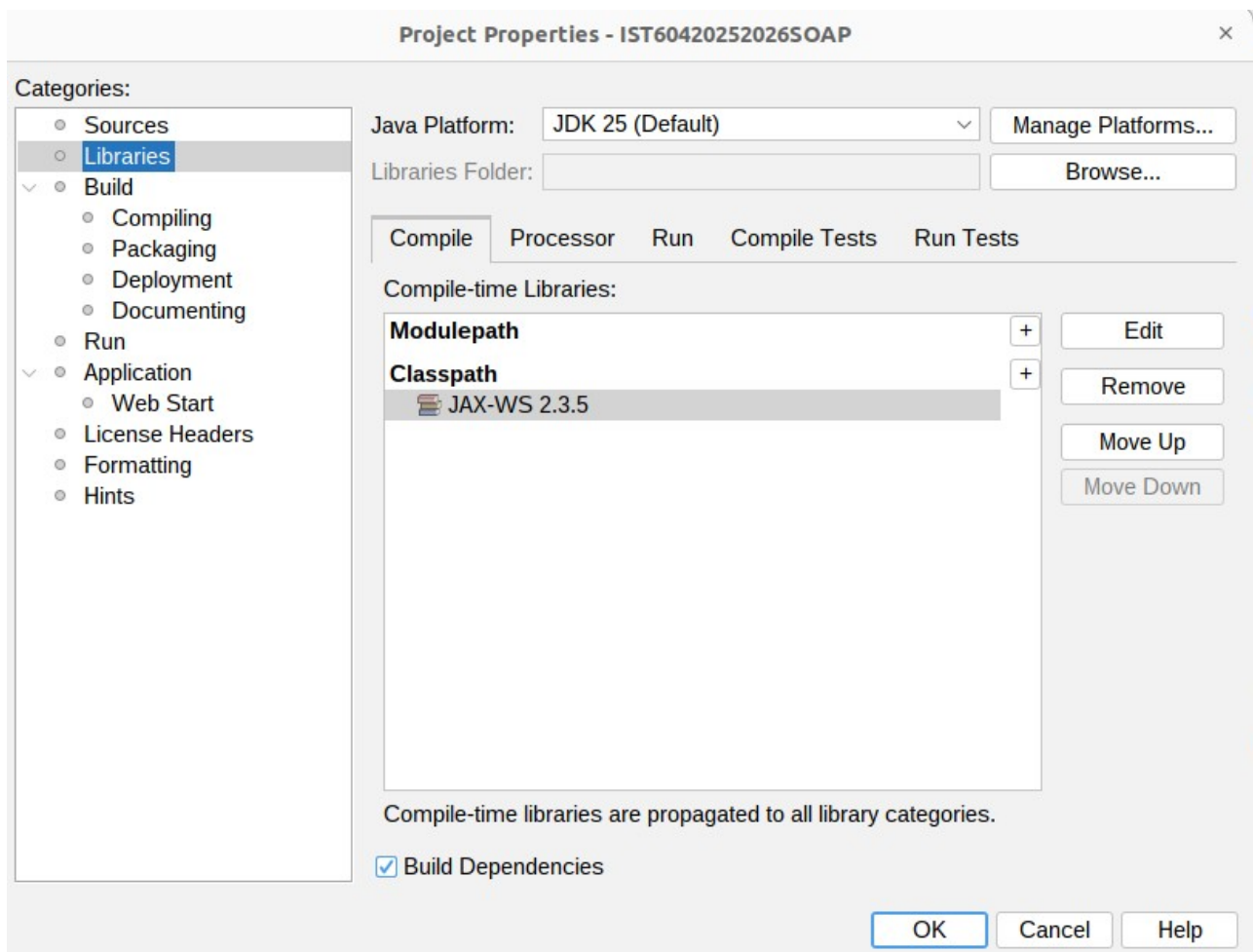
We now convert the TimeServer to a web service, deploy it in core Java, and write 2 clients to consume the web service: one in Perl and the other in Java.

- Start a new project
- Add the JAX-WS library to the **class path** (see illustrations below)



Classpath





TimeServer2Interface.java

```
package JWS;
import javax.jws.WebMethod;
import javax.jws.WebService;
import javax.jws.soap.SOAPBinding;
@WebService
@SOAPBinding(style = SOAPBinding.Style.RPC)
public interface TimeServer2Interface {
    @WebMethod
    String getTimeAsString();
    @WebMethod
    long getTimeAsElapsed();
}
```

TimeServer2Implementation.java

```
package JWS;
import java.util.Date;
import javax.ws.WebService;
@WebService(endpointInterface = "JWS.TimeServer2Interface")
public class TimeServer2Implementation implements TimeServer2Interface {
    @Override
    public String getTimeAsString() {
        return new Date().toString();
    }
    @Override
    public long getTimeAsElapsed() {
        return new Date().getTime();
    }
}
```

Publishing the web service

- In full production mode, a Java Application Server such as GlassFish, Tomcat, JBoss, or WebSphere might be used
- But in development and even light production mode, a simple Java application can be used (see below)

TimeServer2WS.java – Endpoint

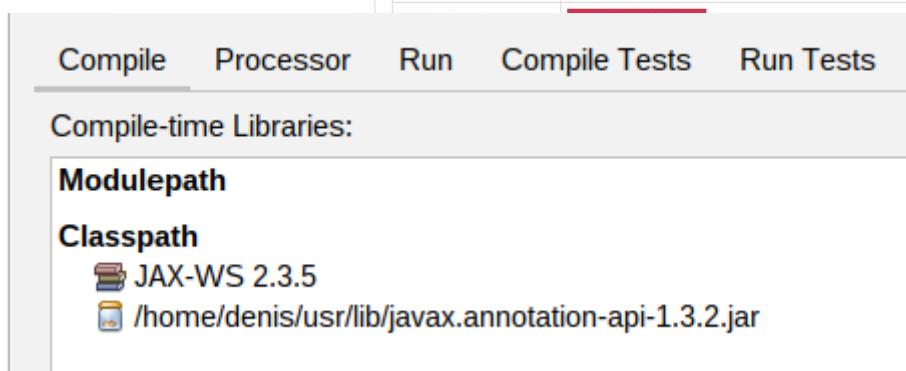
```
package JWS;
import javax.xml.ws.Endpoint;
public class TimeServer2WS {
    public static void main(String[] args) {
        // 1st argument is the publication URL
        // 2nd argument is an SIB instance
        Endpoint.publish("http://127.0.0.1:9873/ts", new TimeServer2Implementation());
    }
}
```

Possible Runtime Error

- When running the application, the following error might occur: `java.lang.NoClassDefFoundError: javax/annotation/Target`
- This error occurs because the `javax.annotation` package is not in the classpath.
- To resolve this, you need to add the `javax.annotation-api` dependency to your project.
- You can find the `javax.annotation-api` dependency in the Maven Repository.
- The `javax.annotation-api` dependency is available in version 1.3.2.



Testing



- URL published in the Java TimeServerPublisher application is <http://127.0.0.1:9873/ts> (equivalent to <http://localhost:9873/ts>)
 - use this URL to access the web server



Web Services

Endpoint	Information
Service Name: { http://JWS/ }TimeServer2ImplementationService	Address: http://127.0.0.1:9873/ts
Port Name: { http://JWS/ }TimeServer2ImplementationPort	WSDL: http://127.0.0.1:9873/ts?wsdl
	Implementation class: JWS.TimeServer2Implementation

The WSDL Document

- We can test the deployed service by opening a browser and viewing the WSDL document, which is an automatically generated service contract
 - <http://127.0.0.1:9873/ts?wsdl> or <http://localhost:9873/ts?wsdl>

```

<!--
  Published by JAX-WS RI (https://github.com/eclipse-ee4j/metro-jax-ws). RI's version is JAX-WS RI 2.3.5 git-revision#7ddc91f.
-->
<!--
  Generated by JAX-WS RI (https://github.com/eclipse-ee4j/metro-jax-ws). RI's version is JAX-WS RI 2.3.5 git-revision#7ddc91f.
-->
<definitions targetNamespace="http://JWS/" name="TimeServer2ImplementationService">
  <types/>
  <message name="getTimeAsString"/>
  <message name="getTimeAsStringResponse">
    <part name="return" type="xsd:string"/>
  </message>
  <message name="getTimeAsElapsed"/>
  <message name="getTimeAsElapsedResponse">
    <part name="return" type="xsd:long"/>
  </message>
  <portType name="TimeServer2Interface">
    <operation name="getTimeAsString">
      <input wsam:Action="http://JWS/TimeServer2Interface/getTimeAsStringRequest" message="tns:getTimeAsString"/>
      <output wsam:Action="http://JWS/TimeServer2Interface/getTimeAsStringResponse" message="tns:getTimeAsStringResponse"/>
    </operation>
    <operation name="getTimeAsElapsed">
      <input wsam:Action="http://JWS/TimeServer2Interface/getTimeAsElapsedRequest" message="tns:getTimeAsElapsed"/>
      <output wsam:Action="http://JWS/TimeServer2Interface/getTimeAsElapsedResponse" message="tns:getTimeAsElapsedResponse"/>
    </operation>
  </portType>
  <binding name="TimeServer2ImplementationPortBinding" type="tns:TimeServer2Interface">
    <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="rpc"/>
    <operation name="getTimeAsString"></operation>
    <operation name="getTimeAsElapsed">
      <soap:operation soapAction=""/>
      <input>
        <soap:body use="literal" namespace="http://JWS/" />
      </input>
      <output>
        <soap:body use="literal" namespace="http://JWS/" />
      </output>
    </operation>
  </binding>
  <service name="TimeServer2ImplementationService">
    <port name="TimeServer2ImplementationPort" binding="tns:TimeServer2ImplementationPortBinding">
      <soap:address location="http://127.0.0.1:9873/ts"/>
    </port>
  </service>
</definitions>

```


3.3.1.3 Time Server Clients

A Perl client for the web service (client.pl)

```
#!/usr/bin/perl -w
use SOAP::Lite;
my $url = 'http://127.0.0.1:9873/ts?wsdl';
my $service = SOAP::Lite->service($url);
print "\nCurrent time is: ", $service->getTimeAsString();
print "\nElapsed milliseconds from the epoch: ", $service->getTimeAsElapsed();
```

Run the Perl script: perl client.pl

```
Current time is: Sun Apr 12 22:40:49 WAT 2026
Elapsed milliseconds from the epoch: 1776030049595(base)
```

A Java client for the web service (TimeClient.java)

```
package JWS;
import java.net.MalformedURLException;
import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;
public class TimeClient {
    public static void main(String[] args) throws MalformedURLException {
        URL url = new URL("http://localhost:9873/ts?wsdl");
        //1st arg is the service URI
        //2nd is the service name published in the WSDL
        QName qname = new QName("http://JWS/",
            "TimeServer2ImplementationService");
        // Create, in effect, a factory for the service.
        Service service = Service.create(url, qname);
        // Extract the endpoint interface, the service "port".
        TimeServer2Interface eif = service.getPort(TimeServer2Interface.class);
        System.out.println(eif.getTimeAsString());
        System.out.println(eif.getTimeAsElapsed());
    }
}
```

Run the Java client

```
Sun Apr 12 22:46:20 WAT 2026
1776030380009
```

3.3.2 Example 2: Hello World Web JAX-WS, RPC Style

4 files involved

1. SEI: Service Endpoint Interface – HelloWorldInterface.java

2. SIB: Service Implementation Bean – HelloWorldImplementation.java
3. Endpoint publisher: HelloWorldWS.java
4. HelloWorldClient.java

HelloWorldInterface.java

```
package JWS;
import javax.jws.WebMethod;
import javax.jws.WebService;
import javax.jws.soap.SOAPBinding;
@WebService
@SOAPBinding(style = SOAPBinding.Style.RPC)
public interface HelloWorldInterface {
    @WebMethod String getHelloWorldAsString(String name);
}
```

HelloWorldImplementation.java

```
package soap.ws3;
import javax.jws.WebService;
@WebService(endpointInterface = "soap.ws3.HelloWorldInterface")
public class HelloWorldImplementation implements HelloWorldInterface{
    @Override
    public String getHelloWorldAsString(String name){
        return "Hello " + name;
    }
}
```

HelloWorldWS.java

```
package JWS;
import javax.xml.ws.Endpoint;
public class HelloWorldWS {
    public static void main(String[] args) {
        Endpoint.publish("http://127.0.0.1:9873/ws/hello", new
        HelloWorldImplementation());
    }
}
```

HelloWorldClient.java

```

package JWS;
import java.net.MalformedURLException;
import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;
public class HelloWorldClient {
    public static void main(String[] args) throws MalformedURLException {
        URL url = new URL("http://localhost:9873/ws/hello?wsdl");
        //1st arg is the service URI
        //2nd is the service name published in the WSDL
        QName qname = new QName("http://JWS/",
"HelloWorldImplementationService");
        Service service = Service.create(url, qname);
        HelloWorldInterface hello = service.getPort(HelloWorldInterface.class);
        System.out.println(hello.getHelloWorldAsString("Bob"));
    }
}

```

Run the Java client

Hello Bob

3.3.3 Example 3: Calculator JAX-WS, RPC Style

4 files involved

1. SEI: Service Endpoint Interface – CalculatorInterface.java
2. SIB: Service Implementation Bean – CalculatorImplementation.java
3. Endpoint publisher: CalculatorWS.java
4. CalculatorClient.java

CalculatorInterface.java

```

package JWS;
import javax.jws.WebMethod;
import javax.jws.WebService;
import javax.jws.soap.SOAPBinding;
@WebService
@SOAPBinding(style = SOAPBinding.Style.RPC)
public interface CalculatorInterface {
    @WebMethod
    int add(int a, int b);
}

```

CalculatorImplementation.java

```
package JWS;
import javax.jws.WebService;
@WebService(endpointInterface = "CalculatorInterface")
public class CalculatorImplementation implements CalculatorInterface {
    @Override
    public int add(int a, int b) {
        return a + b;
    }
}
```

CalculatorWS.java

```
package JWS;
import javax.xml.ws.Endpoint;
public class CalculatorWS {
    public static void main(String[] args) {
        // The service is now live at this URL
        Endpoint.publish("http://localhost:8081/ws/calculator", new
        CalculatorImplementation());
        System.out.println("Service is running at
        http://localhost:8081/ws/calculator?wsdl");
    }
}
```

```
package JWS;
import java.net.MalformedURLException;
import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;
public class CalculatorClient {
    public static void main(String[] args) throws MalformedURLException {
        URL url = new URL("http://localhost:8081/ws/calculator?wsdl");
        QName qname = new QName("http://JWS/",
"CalculatorImplementationService");
        Service service = Service.create(url, qname);
        CalculatorInterface calc =
service.getPort(CalculatorInterface.class);
        System.out.println("Result of 10 + 25 = " + calc.add(10, 25));
    }
}
```

Run the Java client

Result of 10 + 25 = 35

3.4 The Role of the WSDL File

- In JAX-WS, the WSDL file acts as the formal "contract" between the service provider (web service) and the consumer (client).
- Key roles
 - Service Roadmap: It specifies exactly what functions (operations) are available, such as add or subtract in a calculator service
 - Data Structure Definition: It defines the types of data the service expects and returns (e.g., integers, strings, or complex objects) using XML Schema (XSD)
 - Connectivity Details: It provides the exact network address (URL) where the service is hosted and the protocol used (typically SOAP over HTTP)
 - Automation: Because it is machine-readable, tools like **wsimport** use it to automatically generate Java "stubs," allowing developers to call the service like a local method.

3.4.1 wsgen and wsimport

- These are command-line tools used to generate the necessary supporting code (artifacts) for creating or consuming a web service
- Primary difference lies in the **direction** of the development approach

- **wsgen (Bottom-Up)**
 - Use this tool when you have already written your Java code and want to turn it into a web service
 - Input: A compiled Java class (the implementation bean) annotated with `@WebService`
 - Output: JAXB classes used for XML data binding and optionally a WSDL file (using the `-wsdl` flag)
 - Purpose: It generates the artifacts required by the JAX-WS runtime to deploy your service
- **wsimport (Top-Down)**
 - Use this tool when you have an existing WSDL (Web Service Description Language) file and need to build a client implementation from it
 - Input: A WSDL file (either local or via a URL)
 - Output: The Service Endpoint Interface (SEI), Service class, and JAXB data beans
 - Purpose: It generates client stubs that allow you to call the web service as if it were a local Java method
- Comparison table: `wsgen` vs `wsimport`

Feature	wsgen	wsimport
Approach	Bottom-Up (Code first)	Top-Down (Contract first)
Primary Input	Compiled <code>.class</code> files	<code>.wsdl</code> file
Typical User	Service Provider	Service Consumer (Client)
Key Output	JAXB classes, WSDL	Client stubs (SEI, Service)

- Note: Since Java 6, the JAX-WS runtime can often generate the classes generated by `wsgen` dynamically at runtime when you publish an endpoint, making manual `wsgen` calls less common for simple testing
 - Hence, `wsgen` is normally used when you need a physical WSDL file before deployment
 - For example, to share with a client developer or to package within a specific archive

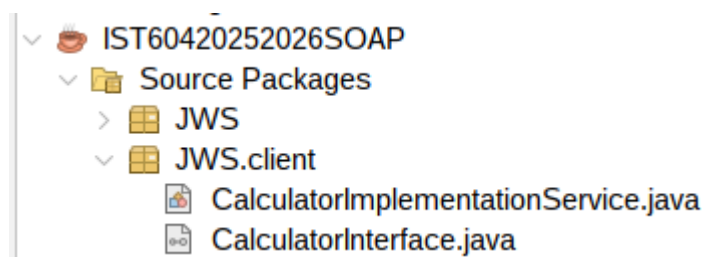
3.4.2 Using `wsimport`

- In the client programs we have written so far, we took advantage of the web service code we had written earlier, e.g., `CalculatorImplementationService` in calculator web service
- In general, the client developer does not have access to the web service code, and so needs the `wsdl` file to be able to develop client code
- `wsimport` Scenario: Consuming the Service

- You are a client developer. Someone else has already hosted the Calculator Service. You have the URL to their WSDL and want to write a Java program to call their add method without manually writing SOAP XML

Java Code (The Client Application)

- Steps
 - Generate Classes: Use the command-line tool `wsimport` (located in your JDK's bin folder)
 - `wsimport -keep -verbose http://example.com`
 - Invoke Service: Once classes are generated, instantiate the service class and get the port to call methods
 - `MyService service = new MyService();`
 - `MyPort port = service.getMyPort();`
 - `String response = port.someOperation("data");`
 - For the calculator web service above
 - From the wsdl file, we notice the following:
 - The service is called `CalculatorImplementationService`
 - Port Name: `CalculatorImplementationPort`
 - Location: <http://localhost:8081/ws/calculator>
 - We however, do not need to track all these, like we did in the earlier client programs
 - Instead, we use the `wsimport` utility to generate stub classes that our client programs can call
- Building the client program for the calculator web service
 - **wsimport**
 - `wsimport -keep -s /home/denis/NetBeansProjects/IST60420252026SOAP/src/JWS -p client http://localhost:8081/ws/calculator?wsdl`
 - `-keep`: Keeps the generated Java source files instead of just the compiled ones
 - `-s src`: Specifies the directory where the source code files should be placed (e.g., your project's src folder).
 - `-p`: Defines the target package name for the generated classes, overriding the default namespace mapping.
 - `http://.../service?wsdl`: The URL (or local file path) of the WSDL file.



- Notice that package client created under package JWS
- **Steps in building the client**
 - // 1. Initialize the generated Service class

- CalculatorService service = new CalculatorService();
- // 2. Get the Port (the actual interface to call methods)
 - CalculatorInterface proxy = service.getCalculatorPort();
- // 3. Call the web service like a local Java method
 - int result = proxy.method(parameter 1, parameter 2, ...);

CalcClient.java

```
package JWS.client;
public class CalcClient {
    public static void main(String[] args) {
        CalculatorImplementationService service = new
        CalculatorImplementationService();
        CalculatorInterface port =
        service.getCalculatorImplementationPort();
        int result = port.add(10, 20);
        System.out.println("Result from Web Service: " + result);
    }
}
```

Result from Web Service: 30

3.5 Document Style

- Recall that the Service Endpoint Interface (SEI) for the examples we have seen so far contain the annotation: @SOAPBinding(style = Style.RPC)
- This annotation requires that the service use only very simple types such as string and integer
- While RPC style mimics a function call, the Document style is the industry standard for sending full XML documents as messages
- Document style requires more setup, as illustrated below (Publishing the Web Service)
- For richer data types, we use Style.DOCUMENT to replace Style.RPC, or omit it altogether since Style.DOCUMENT is the default

3.5.1 Example 1: Order Processing Service, Document Style

The service accepts an "Order" object and returns a confirmation status.

4 files involved

- SEI: Service Endpoint Interface – OrderService.java

- SIB: Service Implementation Bean – OrderServiceImpl.java
- Endpoint publisher: OrderPublisher.java
- Purchasing application (Client): OrderClient.java

OrderService.java

```
package JWS;
import javax.xml.ws.WebMethod;
import javax.xml.ws.WebService;
import javax.xml.ws.soap.SOAPBinding;
@WebService
@SOAPBinding(style = SOAPBinding.Style.DOCUMENT) // Industry Standard
public interface OrderService {
    @WebMethod String placeOrder(String item, int quantity);
}
```

OrderServiceImpl.java

```
package JWS;
import javax.xml.ws.WebService;

@WebService(endpointInterface = "JWS.OrderService")
public class OrderServiceImpl implements OrderService {
    @Override
    public String placeOrder(String item, int quantity) {
        System.out.println("Processing order for: " + item + " x" + quantity);
        return "SUCCESS: Order #" + (int)(Math.random() * 1000) + " confirmed.";
    }
}
```

OrderPublisher.java

```
package JWS;
import javax.xml.ws.Endpoint;
public class OrderPublisher {
    public static void main(String[] args) {
        String url = "http://localhost:8888/ws/orders";
        Endpoint.publish(url, new OrderServiceImpl());
        System.out.println("Order Service is live at: " + url + "?wsdl");
    }
}
```

When you compile the publisher (web service), you may see a warning message from the JVM

This message is due to changes that have been made from Java 9 onwards

We ignore the warning message

```

package JWS;
import java.net.MalformedURLException;
import java.net.URL;
import javax.xml.namespace.QName;
import javax.xml.ws.Service;
public class OrderClient1 {
    public static void main(String[] args) throws MalformedURLException {
        // 1. Point to the WSDL URL
        URL wsdlUrl = new URL("http://localhost:8888/ws/orders?wsdl");
        // 2. Identify the service using the TargetNamespace and Service Name
        from the WSDL
        QName qname = new QName("http://JWS/", "OrderServiceImplService");
        // 3. Create the Service factory
        Service service = Service.create(wsdlUrl, qname);
        // 4. Extract the Port (the proxy object)
        OrderService orderService = service.getPort(OrderService.class);
        // 5. Invoke the method
        String response = orderService.placeOrder("Laptop", 2);
        System.out.println("Server Response: " + response);
    }
}

```

Server Response: SUCCESS: Order #719 confirmed.

##Alternatively, the OrderClient doesn't need to know the underlying XML; it uses the OrderService interface as a "proxy" to communicate with the remote server

- Run wsimport to generate client stubs from the WSDL
 - wsimport -keep -s /home/denis/NetBeansProjects/IST60420252026SOAP/src/JWS -p JWS.orderclient http://localhost:8888/ws/orders?wsdl

```

package JWS.orderclient;
import java.net.URL;
public class OrderClient2 {
    public static void main(String[] args) {
        // Use the auto-generated service class
        OrderServiceImplService service = new OrderServiceImplService();
        // Get the proxy port
        OrderService port = service.getOrderServiceImplPort();
        // Call the service
        System.out.println(port.placeOrder("Tablet", 5));
    }
}

```

SUCCESS: Order #549 confirmed.